

# From Maze to Arena

The whole story in one read — where the Leaper began, where it stands today, where we're taking it, and how the pieces of the game fit together.

---

*This is not a list of answers. It's a walkthrough — a single, connected story of a small robot that learned to move, and the game we're now going to build around it. Read it start to finish and you'll hold the whole architecture in your head at once.*

We'll move through four simple stages: **where we started, where we are now, where we're going**, and **how we'll actually do it**. By the end, every part of the game — the trained brain, the browser, the shooting, the damage — should sit in one clear picture.

## FIRST, THE SHORT WORDS SPELLED OUT

A handful of short forms come up throughout. Here they are once, plainly, so the rest reads smoothly:

|      |                                                                                                     |
|------|-----------------------------------------------------------------------------------------------------|
| RL   | <b>Reinforcement Learning</b> — teaching by trial and error over many thousands of attempts.        |
| PPO  | <b>Proximal Policy Optimization</b> — the specific RL method used to train the brain.               |
| MLP  | <b>Multi-Layer Perceptron</b> — the simple, standard kind of network the brain is made of.          |
| LSTM | <b>Long Short-Term Memory</b> — an add-on that gives a network memory. We tried it; it didn't help. |
| ONNX | <b>Open Neural Network Exchange</b> — a universal brain file that web browsers can read.            |
| FOV  | <b>Field Of View</b> — how wide the robot can see; here a broad 270°.                               |
| FSM  | <b>Finite State Machine</b> — a way of writing behavior where the robot is in one "mode" at a time. |

## STAGE ONE

# Where we started

The whole project began with a single, stubborn goal: teach a six-legged robot to **walk to a target on its own** — through a cluttered space full of obstacles, without being told the route. We didn't program the movement by hand. We used **RL (Reinforcement Learning)**: the robot tried, failed, and slowly kept whatever worked, hundreds of thousands of times over, until good movement emerged on its own.

That journey had a clear shape, and it's worth seeing all at once, because it's the reason the brain is what it is today. We solved the easy problems first, then spent weeks chasing one stubborn one — and the way we finally cracked it taught us the single most important lesson of the whole project.



**The road to the brain.** Crashing and freezing were solved early. Then one problem held on for weeks — the robot getting lost in dead-ends. We tried making it *think* harder (memory, then a bigger brain); neither helped. What finally worked was letting it see better — sharper eyes. That, plus steadier confidence, gave us today's champion.

The lesson buried in that timeline shapes everything ahead: **the robot's limits came from what it could see, not how hard it could think.** We gave it memory (an **LSTM, Long Short-Term Memory** add-on) three separate times, expecting it to stop repeating dead-ends — and it never beat the simpler brain. It only broke through when we doubled its eyesight. Keep that in your pocket; it explains why the game leans on the brain for one job only, and hands everything else to simpler tools.

## STAGE TWO

# Where we are now

Today we hold two things. The first is a **finished, capable brain** — the champion, an **MLP (Multi-Layer Perceptron)** trained with **PPO** that reaches its target about **89%** of the time, steering confidently around obstacles with a wide 270° **FOV (Field Of View)**. It is small, fast, and — by design — has no memory, which turns out to make it much easier to carry into a browser.

The second is a **web project already written in JavaScript**, using a tool called **Three.js** to draw the robot and its world in 3D inside an ordinary browser. So the graphics foundation is already built. There is no "switch to another language" ahead of us — the game is already in the language of the web.

But there's one honest gap, and naming it is the whole reason for the next stage. Right now, the browser doesn't actually *run* the brain. It plays back a **recording** — a large file that

Python made earlier, like a video of the robot moving. It's perfect for studying the training, but you can't play against a recording. It never reacts. That single gap — *a recording that can't respond* — is exactly what we're about to close.

### STAGE THREE

## Where we're going

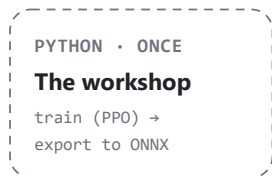
The destination is a **real, playable game**. You control a player in the arena. The Leaper is your opponent: it chases you using its trained brain, it shoots at you, and when it's badly hurt it turns and flees. You shoot back — and because the Leaper is built from six separate legs, you can see it come apart as you damage it: a leg blows off, its walk turns to a limp, sparks fly, until it finally collapses.

We're deliberately starting small and honest: **one human versus one Leaper**. That's the version that proves the brain truly works in a live game, and it's genuinely fun on its own. A friend joining on the same screen, and — much later — online play, are optional adventures for once the core game feels good. We don't need them to have something worth sharing.

### STAGE FOUR

## How the game fits together

Here is the architecture — the shape of how all of it works. The trick that makes this manageable is a clean division of labour: **the trained brain does the one thing it's brilliant at (moving), and simple hand-written rules do the fighting**. Nothing is asked to do a job it wasn't built for.



leaper.onnx

THE BROWSER • GAME LOOP • ~60x PER SECOND

#### Movement brain

the trained MLP (via ONNX)  
“where do I step?”

#### Combat brain

hand-written rules  
“shoot, or flee?”

#### Move the robot & draw the frame

apply the decisions • fly the bullets • check hits • Three.js renders it all  
...then start the loop over again

**One picture of the whole thing.** Python hands over a single file, once. Then the browser loops forever: the **movement brain** decides where to step, the **combat brain** decides whether to shoot or flee, and the loop applies both, moves the bullets, checks the hits, and draws the frame — sixty times a second.

## The one careful step: waking the brain up

Getting the brain from Python into the browser is the crux. We export it once to an **ONNX (Open Neural Network Exchange)** file — the universal brain-format browsers can read — and run it live with a small library. The brain expects to be handed **26 numbers** each frame (16 vision-ray distances plus 10 facts about itself and its goal), and it answers with **2** (how hard to go, how sharply to turn). The single make-or-break detail is that those 26 numbers must be built in JavaScript in *exactly* the same order and scale that Python used. Get that right and the live brain moves just like the trained one; get it subtly wrong and it behaves as if dizzy. That faithfulness is the real work of this step.

### WHY THE SPLIT MATTERS

Remember the lesson from Stage One — the brain is a brilliant *mover* but was never taught to *fight*. So we don't ask it to. It handles movement; a few plain rules handle combat. This is the same wisdom that ended the memory experiments: use each tool for the job it's actually good at, and nothing for a job it isn't.

# Start simple, upgrade only if needed

The Leaper's fighting decisions need no training at all — we simply write them. And we'll write them in the simplest form that works, reaching for something fancier only when the game actually shows us we need it. This is a two-step plan on purpose.

## Step one — the behavior tree (a plain checklist)

A **behavior tree** is exactly the "if this, then that" idea you already have. It runs top-to-bottom every single frame and does the first thing that's true:

1. *Am I hurt?* → *flee*.
2. *Is the player in range?* → *shoot*.
3. *Otherwise* → *chase (hand control to the movement brain)*.

That's the whole combat brain, and for most of the game it's all we'll ever need. Notice how cleanly it hands off: the first two lines are the fighting; the last line simply gives the wheel back to the trained brain to do the chasing. Your months of RL work power that final line.

## Step two — the state machine (only if the checklist isn't enough)

A plain checklist has one blind spot: it **forgets what it was just doing**. It decides fresh every frame, with no sense of "I'm in the middle of something." Two situations expose that, and both are where a **state machine (FSM, Finite State Machine)** earns its place. The difference in one line: a checklist decides from scratch each frame; a state machine puts the Leaper *in a mode* and makes it *stay there* until a specific thing changes.

**The twitching problem.** Imagine you dance right at the edge of shooting range, stepping in and out. The plain checklist flickers — shoot, chase, shoot, chase — many times a second, because each frame it forgets it was just shooting. A state machine fixes this by committing: "I've entered *Shooting* mode, and I'll stay in it until the player is *clearly* out of range" — so it stops flip-flopping over a tiny wobble.

**The longer-action problem — your example.** This was the sharp point you made, and it's the best reason of all. Some actions take *time* and must be allowed to play out. Picture the Leaper **reloading**: once it starts, it should be locked into reloading for a couple of seconds, even if you run at it. Or a **charge attack** that goes wind-up → dash → recover, three phases in order. A checklist, deciding fresh every frame, has no natural way to "stay committed to

something that takes time." A state machine does it effortlessly, because it simply remembers "I'm in *Reloading* mode right now, and I don't leave until the timer's done."

#### THE PLAN, IN A SENTENCE

Build the behavior-tree checklist first — it's the clearest place to start and handles most of the game. The day the Leaper starts twitching at the edge of range, or the day we add an action that takes time (a reload, a charge), that's our signal to give *that one behavior* the small memory of a state machine. Not before. The two aren't rivals — the common setup is a state machine for the big modes with a little checklist living inside each.

#### THE SHAPE OF IT ALL

So that's the full arc in one breath. We **started** by teaching a robot to move with reinforcement learning, and learned — the hard way, through the memory experiments — that its power is in *moving*, not thinking harder. We **are** now holding that finished brain and a web project that only replays it. We're **going** to turn that into a real one-on-one game where the Leaper hunts, shoots, and can be torn apart leg by leg. And we'll **do it** by carrying the brain into the browser through a single ONNX file, letting it drive movement inside a sixty-times-a-second loop, and layering plain combat rules on top — a behavior-tree checklist first, a state machine only where the Leaper needs to remember what it's doing. One careful step, then a series of small, visible, playable ones. That's the road from maze to arena.